PersonalAgent

A Technical Guide to the Local Research RAG Assistant

Project purpose

PersonalAgent is a privacy-oriented question-answering application for research papers. It accepts local PDF files, converts their text into searchable vector representations, retrieves the most relevant excerpts for a question, and asks a locally hosted language model to answer using those excerpts. The result is a small but complete Retrieval-Augmented Generation (RAG) system.

The design has two phases: an offline indexing phase handled by ingest.py, and an interactive question-answering phase handled by app.py and rag.py.

Report scope

This report describes the implementation present in the project files, including its architecture, data flow, technology stack, algorithms, configuration, setup process, privacy model, limitations, and practical improvement opportunities.

1. Executive Overview

The application behaves like a private research librarian. The user supplies papers, the system builds a local index, and questions are answered from retrieved passages instead of from the language model's general memory alone. This is important because a language model by itself may produce plausible but unsupported answers. RAG adds a document-retrieval step and gives the model a bounded evidence set.

Core capabilities

• Reads text-based PDF research papers from the papers/ directory.

• Preserves the source paper name and page number for every indexed chunk.

• Uses semantic embeddings rather than exact keyword matching.

• Stores the index persistently in the local chroma_db/ directory.

• Uses Ollama to generate answers without sending paper content to a cloud API.

• Displays retrieved paper and page sources below each answer.

High-level architecture

Layer	Component	Responsibility
User interface	Streamlit / app.py	Chat input, model selection, conversation history, and source display.
Application logic	rag.py	Embeddings, similarity search, prompt construction, and Ollama requests.
Ingestion	ingest.py	PDF extraction, page tracking, chunking, embedding, and indexing.
Vector storage	ChromaDB	Persistent collection of vectors, text chunks, and paper/page metadata.
Embedding model	all-MiniLM-L6-v2	Converts both document chunks and questions into comparable numeric vectors.
Generation	Ollama + llama3.1	Runs the language model locally and generates a grounded response.

The system is intentionally modular: ingestion can be rerun independently, retrieval can be tested independently of the UI, and the UI delegates domain behavior to rag.py.

2. Architecture and Data Flow

There are two connected pipelines. The first transforms documents into a searchable knowledge base. The second transforms a user question into a grounded answer.

2.1 Indexing pipeline

• File discovery: ingest.py scans papers/ for filenames ending in .pdf.

• PDF opening: PyMuPDF opens each document and iterates through its pages.

• Text extraction: page.get_text() returns the text layer. Empty pages are discarded.

• Chunking: each page is split into groups of 700 words with 100 words of overlap.

• Embedding: all-MiniLM-L6-v2 maps each chunk to a dense vector.

• Persistence: ChromaDB stores the vector, original text, and metadata under a stable chunk ID.

2.2 Question pipeline

• The user enters a question in Streamlit.

• rag.search() embeds the question with the same embedding model.

• ChromaDB returns the five nearest document chunks by default.

• rag.build_prompt() combines those chunks with strict answer instructions.

• requests.post() sends the prompt to Ollama's local HTTP endpoint.

• The generated answer and retrieved sources are displayed in the chat interface.

2.3 Storage model

ChromaDB uses a persistent client pointed at chroma_db/. The collection is named papers. Each record contains an ID, an embedding, the original chunk text, and metadata. Because the database is persistent, the application does not need to recreate embeddings every time the chat UI starts.

Example record

```
ID:       paper.pdf_p4_c2
Text:     retrieved 700-word excerpt
Metadata: {"paper": "paper.pdf", "page": 4}
Vector:   embedding generated by all-MiniLM-L6-v2
```

3. Technology Stack

Technology	Role
Python	Primary programming language and runtime.
Streamlit	Turns app.py into a browser-based chat application without a separate frontend framework.
PyMuPDF / fitz	Opens PDFs and extracts their text page by page.
Sentence Transformers	Loads the all-MiniLM-L6-v2 embedding model.
all-MiniLM-L6-v2	A compact sentence embedding model suitable for local semantic search.
ChromaDB	Vector database used to store and query embedded chunks.
Ollama	Local model runner exposing an HTTP generation API.
llama3.1	Default local generative model, configurable from the sidebar.
Requests	Python HTTP client used to call Ollama.

Why this stack works

The components cover the full RAG lifecycle while remaining lightweight. PyMuPDF handles document input, Sentence Transformers handles semantic representation, ChromaDB handles retrieval, and Ollama handles generation. Streamlit provides an immediate usable interface. There is no separate cloud backend, API gateway, frontend build system, or hosted database.

Local versus cloud responsibilities

After installation and model downloads, normal operation can be offline. PDF text, embeddings, questions, prompts, and generated answers remain on the computer. Internet access is still required initially to install Python packages and download the embedding and Ollama models.

4. Code Walkthrough

4.1 ingest.py

The constants define the main indexing configuration: PAPERS_FOLDER is papers, DB_FOLDER is chroma_db, CHUNK_SIZE is 700 words, and CHUNK_OVERLAP is 100 words.

extract_pages(pdf_path) returns (page_number, text) pairs. Page numbers are converted from Python's zero-based index to human-readable numbering by adding one. Pages with only whitespace are excluded.

chunk_text(text) splits on whitespace, joins words back into strings, and advances by chunk_size - overlap. The overlap helps retain context when an important sentence falls near a chunk boundary.

main() creates the embedding model, opens a persistent Chroma client, creates or reuses the papers collection, processes every PDF, and calls collection.upsert(). Upsert means rerunning a file with the same generated IDs replaces those records rather than creating exact duplicates.

4.2 rag.py

The module-level variables _embedder and _collection cache expensive resources. _get_embedder() and _get_collection() initialize them only on first use. This avoids repeatedly loading the embedding model or reopening the collection during a Streamlit session.

search(question, top_k=5) encodes the question, calls ChromaDB query(), and converts the result arrays into a simple list of dictionaries. The result format is deliberately UI-friendly: every item has text, paper, and page keys.

build_prompt() creates the grounding boundary. It labels each excerpt with its source and instructs the model to answer only from those excerpts, acknowledge missing evidence, and cite claims.

ask() combines retrieval and generation. It handles an empty database with a friendly message, otherwise posts JSON to Ollama, disables streaming, enforces a five-minute timeout, raises HTTP errors, and returns the generated response with its source chunks.

4.3 app.py

The Streamlit page sets the title, creates a history list in session state, lets the user choose an Ollama model, renders previous messages, accepts a new chat question, calls ask(), and exposes sources in an expander. The session history is temporary and is lost when the session is restarted.

5. RAG Concepts Used

Embeddings

An embedding is a numeric representation of text. Texts with similar meaning tend to be close together in vector space, even when they do not share the same keywords. The project embeds both paper chunks and questions with the same model so they can be compared.

Semantic retrieval

When a user asks a question, ChromaDB compares the question vector with stored chunk vectors and returns the nearest results. The current implementation retrieves five chunks. This is semantic search, not a full keyword search or a hybrid search.

Grounded generation

The retrieved chunks are inserted into the prompt as excerpts. The model is asked to treat them as the only evidence. This reduces unsupported answers, although prompting alone cannot guarantee perfect citation or factual compliance.

Metadata and citations

Metadata connects each chunk back to its source. The prompt includes paper and page labels, and the UI separately renders those labels. This provides useful traceability even though the application does not validate whether every generated citation is accurate.

Chunking tradeoffs

Larger chunks preserve more context but may dilute retrieval precision and consume more prompt space. Smaller chunks improve precision but can remove the surrounding explanation needed to interpret a result. The fixed 700-word and 100-word settings are reasonable starting points, not universal optimums.

6. Installation and Operation

Environment setup

The repository currently contains a .venv directory. On Windows, the interpreter can be used directly without activation:

```
.\.venv\Scripts\python.exe -m pip install -r requirements.txt
```

Ollama must be installed separately. Then download the default model:

```
ollama pull llama3.1
```

If system memory is limited, the README suggests qwen2.5:3b and the model name can be entered in the app sidebar.

Index papers

```
New-Item -ItemType Directory -Force papers\nCopy-Item C:\path\to\paper.pdf papers\n.\.venv\Scripts\python
```

The first ingestion run may download the embedding model. The generated vector database is stored locally in chroma_db/.

Run the interface

```
.\.venv\Scripts\python.exe -m streamlit run app.py
```

Streamlit prints a local URL, normally http://localhost:8501, and opens the application in a browser. Enter a question, select the desired Ollama model, and expand Sources to inspect retrieved paper pages.

Operational sequence

Whenever papers are added or changed, rerun ingest.py before asking questions. The chat app does not watch the papers folder and does not automatically rebuild the index.

7. Limitations, Risks, and Observations

Scanned PDFs: Image-only documents have no text layer, so extract_pages() returns no useful text. OCR is not implemented.

Complex layouts: Tables, multi-column reading order, equations, headers, and footers may be extracted imperfectly.

Stale records: The current ingestion process upserts discovered files but does not remove records for PDFs that were deleted or renamed.

README mismatch: The README says only new files need processing, but ingest.py currently loops over every PDF on every run.

Empty collection: Searching an unindexed Chroma collection may depend on Chroma's behavior for empty queries; the user-facing empty-database check occurs after search().

Model errors: A missing Ollama model, stopped Ollama service, connection failure, or malformed response raises an error from the request path.

Citation validation: The model is instructed to cite sources, but the application does not verify citations against retrieved metadata.

No tests: There is no automated test suite, so ingestion, retrieval, prompt behavior, and UI execution are not regression-tested.

Temporary chat state: Conversation history is kept in Streamlit session state and is not durable or shared between users.

Security and privacy model

The design avoids external inference APIs and does not require API keys. The local Ollama HTTP endpoint is unauthenticated by default, so the machine's local security posture still matters. The application should not be exposed to an untrusted network without adding authentication and request controls.

Performance

Embedding is generally the expensive part of indexing, while generation is usually the slowest interactive step. The embedding model is cached in memory by rag.py, and ChromaDB avoids rebuilding the database at app startup. Larger generation models improve reasoning but require more memory and may respond more slowly.

8. Recommended Improvements

• Fix the Streamlit typo st.markedown to st.markdown and ensure assistant history is appended only after a question is processed.

• Use absolute paths based on the project directory so launching the scripts from another working directory still finds papers/ and chroma_db/.

• Track file hashes or modification times so unchanged PDFs are skipped and changed PDFs are reindexed deliberately.

• Remove stale Chroma records when a source PDF is deleted or renamed.

• Add OCR support for scanned PDFs, for example through a configurable OCR preprocessing step.

• Add a reranker or hybrid keyword-plus-vector retrieval for better precision on technical terminology.

• Make top_k, chunk size, overlap, and model names configurable through a settings file or environment variables.

• Add tests for chunk boundaries, page metadata, prompt construction, empty collections, Ollama failures, and source rendering.

• Handle request exceptions with a clear UI message instead of exposing a raw traceback.

• Validate or post-process generated citations so displayed claims correspond to retrieved sources.

• Add document management controls for listing, rebuilding, and deleting indexed papers.

Suggested testing strategy

Unit tests should first cover chunk_text() with short, exact-boundary, and overlapping inputs. Next, test extract_pages() with a small fixture PDF and assert page numbers. Mock SentenceTransformer, ChromaDB, and requests.post() when testing rag.py so tests remain fast and offline. Finally, run a smoke test that starts Streamlit and confirms a question can render with a mocked or local Ollama service.

Future architecture

The current single-process structure is appropriate for a personal tool. A larger version could separate ingestion into a background job, add a document registry, use a reranking stage, stream Ollama tokens to the UI, and persist conversations. Those changes should come after correctness, source cleanup, and test coverage.

9. Quick Reference

Project files

```
app.py          Streamlit user interface
ingest.py       PDF ingestion and vector indexing
rag.py          Retrieval and Ollama generation
requirements.txt Python dependencies
README.md       Setup and usage notes
papers/         Input PDFs (created on first ingestion)
chroma_db/      Persistent vector database
.venv/          Local Python environment
```

Typical command sequence

```
.\.venv\Scripts\python.exe -m pip install -r requirements.txt\nollama pull llama3.1\n.\.venv\Scripts\pyth
```

Final summary

PersonalAgent is a clear demonstration of a local RAG architecture. Its strongest properties are privacy, simple deployment, traceable source metadata, and a small number of understandable components. Its main gaps are production hardening, OCR, stale-index management, test coverage, and a few runtime/documentation inconsistencies. With those addressed, it can serve as a dependable private research-paper assistant.